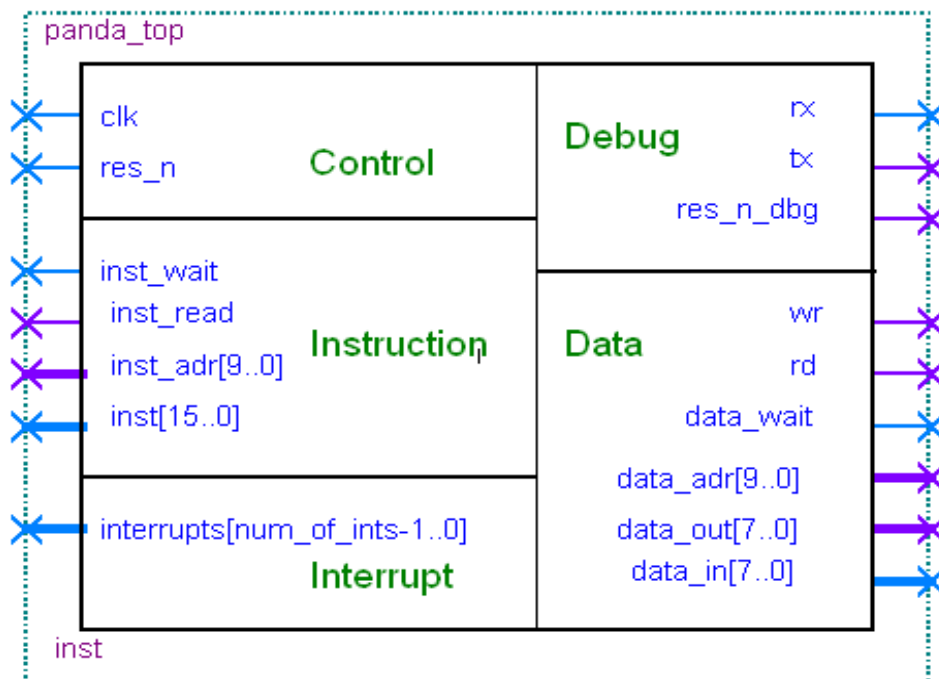




PANDA - Documentation

PANDA – Configurable Softcore Processor



written in VHDL

Logic Solutions Bründler
Winkelbühlrain 10
CH-6043 Adligenswil
Switzerland
Tel. +41 (0)76 408 83 58
E-Mail info@logicsolutions.ch
Internet www.logicsolutions.ch

Table of Contents

1 Legal Notices.....	5
2 Core Documentation.....	6
2.1 Features.....	6
2.2 Architecture Overview.....	7
2.2.1 Reset Values.....	7
2.3 Instruction Set.....	8
2.3.1 Instruction Set Summary.....	8
2.3.2 Indirect Addressing.....	10
2.3.3 Skip Instructions.....	10
2.3.4 Interrupts.....	10
2.3.5 Stack.....	10
2.3.6 Multiplication.....	11
2.3.7 Multiply-Accumulate.....	11
2.4 Interfaces.....	12
2.4.1 Instruction Interface.....	12
2.4.1.1 General Description.....	12
2.4.1.2 Normal Instructon Read.....	13
2.4.1.3 Instruction Read with Wait1.....	13
2.4.2 Data Interface.....	14
2.4.2.1 General Description.....	14
2.4.2.2 Normal Data Read.....	14
2.4.2.3 Data Read with Wait.....	15
2.4.2.4 Normal Data Write1.....	15
2.4.2.5 Data Write with Wait1.....	15
2.4.3 Interrupts.....	15
2.4.4 Debug Interface	15
2.5 How to configure the Core.....	16
2.5.1 Generics.....	16
2.5.1.1 stack_depth.....	16
2.5.1.2 num_of_ints.....	16
2.5.1.3 interrupts_on.....	16
2.5.1.4 indir_on.....	17
2.5.1.5 carry_arith_on.....	17
2.5.1.6 adv_skips_on.....	17
2.5.1.7 mul_on.....	18
2.5.1.8 mac_for_mul.....	18
2.5.1.9 user_stack_on.....	18
2.5.1.10 stack_as_ram.....	18
2.5.1.11 dat_on_sstack.....	19
2.5.1.12 debug_on.....	19
2.5.1.13 clk_frequency.....	19
2.5.2 Package panda_pkg.vhdl.....	20
2.5.2.1 Databus Address Width.....	20
2.5.2.2 Instructon Address Width.....	20
2.5.2.3 System Stack Width.....	20
2.5.2.4 Biggest Register Width.....	20
2.5.3 Expanding the Core to 16-Bit Wordwidth.....	21
2.5.3.1 Constants in panda_pkg.vhdl.....	21
2.5.3.2 Register Widths.....	21
2.5.3.3 Assembler.....	21
2.6 Connecting Peripherals.....	22

2.6.1 Instruction ROM.....	22
2.6.2 Data Bus Block.....	22
2.6.2.1 data_in Multiplexer.....	22
2.6.2.2 Chip Select Signals.....	23
2.6.2.3 Data RAM Write Signal.....	23
2.6.3 Own Peripherals.....	23
2.7 Files.....	23
2.8 Synthesis Results.....	24
2.8.1 Altera.....	24
2.8.1.1 MAX II CPLD (EPM1270GF256C3).....	24
2.8.1.2 Cyclone II FPGA (EP2C5Q208C7).....	24
2.8.2 Xilinx.....	25
2.8.2.1 Spartan3 FPGA (XC3S200).....	25
2.8.3 Lattice.....	26
2.8.3.1 MachXO CPLD (LCMXO2280E-5FT256C).....	26
2.8.3.2 XP2 FPGA (LFXP3E-4208C).....	26
3 Software Installation Guide.....	27
3.1 Assembler Installation.....	27
3.2 Editor Installation.....	27
4 Software User Guide.....	30
4.1 Software development Flow.....	30
4.2 Editor User Guide.....	30
4.2.1 Enabe Syntax Highlighting.....	30
4.2.2 Assemble a Source-File.....	31
4.2.3 View the Listing.....	31
4.2.4 Creating ROM-Files.....	32
4.3 Assembler Syntax.....	33
4.3.1 Definitions and Directives.....	33
4.3.2 Basic Conventions.....	33
4.3.2.1 Comments.....	33
4.3.2.2 Number Formats.....	33
4.3.2.3 Operands for fixed Numbers.....	33
4.3.2.4 Segments.....	33
4.3.3 Declaring Peripheral Registers.....	34
4.3.4 Declaring Variables.....	34
4.3.5 Declaring Constants.....	34
4.3.6 Macros.....	35
4.3.7 Code Syntax.....	35
4.3.8 Interrupts.....	36
4.3.9 Include File.....	36
4.4 On-Chip-Debugger.....	36
4.4.1 Hardware Connections.....	36
4.4.2 Commands.....	37
4.4.3 Interpreting the Response.....	37

Illustration Index

Illustration 1: Architecture Overview.....	7
Illustration 2: inst_wait generation for synchronuous ROMs.....	12
Illustration 3: Normal Instruction Read.....	13
Illustration 4: Instruction Read with Wait.....	13
Illustration 5: Normal Data Read.....	14
Illustration 6: Data Read with Wait.....	15
Illustration 7: Normal Data Write.....	15
Illustration 8: Data Write with Wait.....	15

Table Index

Table 1: Reset Values of internal Registers.....	7
Table 2: Instruction Set Summary.....	8
Table 3: Signals of the Instruction Interface.....	12
Table 4: Signals of the Data Interface.....	14
Table 5: Constant Values for 16-Bit.....	21
Table 6: Rom Connection.....	22
Table 7: Peripheral Connection.....	23
Table 8: Debugger RS-232 Connection Settings.....	36
Table 9: Debugger Commands.....	37
Table 10: Response of Debugging Module.....	37

Abstract

This document is a full documentation of the PANDA softcore processor. It contains the design documentation, information about needed peripherals (ROM, RAM, BUS), software installation- and user-guide.

The goal of this design is to provide a widely configurable, vendor independent, small softcore processor and a well fitting software IDE (for assembler). The design is not optimized for speed but for size, so the processor is also synthesizable on newer CPLD familys (e.g. MAXII, MachXO) which have small memory ressources on chip.

The result is a microprocessor that fits into less than 170 logic elements and runs up to more than 110MHz on an Altera MAX II Device in its smallest configuration. Each instruction needs two to three clock-cycles to be executed. An on-chip-debugger can optionally be implemented too.

The top module is available in two different versions. One has std_logic I/Os, the other one has std_ulogic I/Os.

All peripherals must be attached by the user. This makes the processor core easy to addapt on given problems. Another advantage is, that there are only needed peripherals synthesized. This saves device ressources.

1 Legal Notices

The processor and all components of the builder software are under LGPL license and can be used freely for any purposes.

Third party SW such as the HASM is not included in this licesne. HASM can be bought from www.pohltroonik.de. For non-commercial purposes, HASM is free.

For any sales or legal questions please write an e-mail to info@logicsolutions.ch

2 Core Documentation

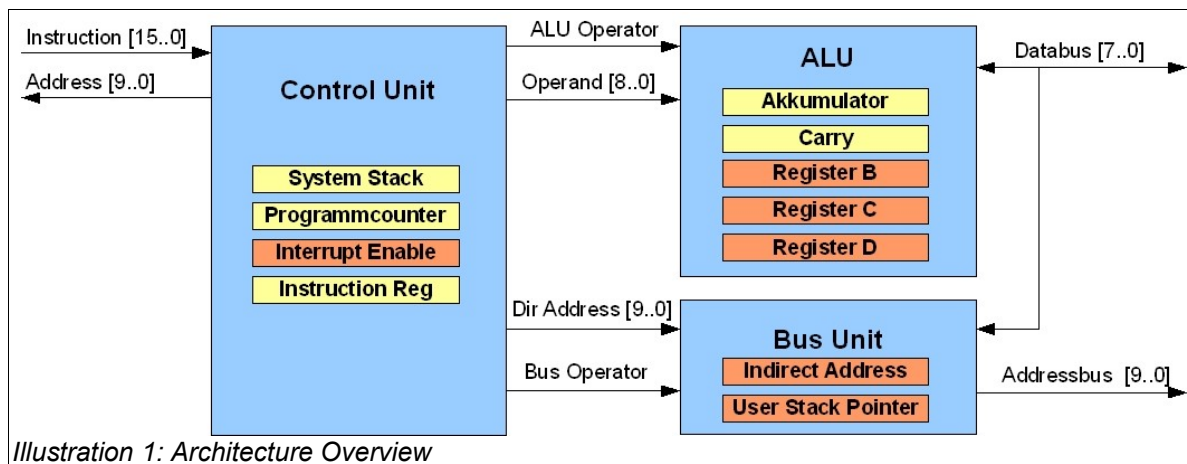
This part of the document describes the cpu core with all its interfaces, internal registers, instructions and other relevant information.

2.1 Features

- RISC Architecture
- 8bit Datapath
- Widely configurable by generics
 - Configurable stack-depth (Default is 6)
 - Number of interrupts can be changed (Default is 1)
 - Interrupt-support can be enabled / disabled
 - Indirect addressing can be enabled / disabled
 - Carry arithmetic can be enabled / disabled
 - Some useful but not necessary skip instructions can be enabled / disabled
 - Single cycle multiplier can be enabled / disabled
 - Single cycle multiplier can be replaced by a multiply-accumulate instruction
 - User stackpointer can be enabled / disabled
 - Instruction stack can be implemented either as FF-Array (CPLDs) or as RAM (FPGAs)
 - System Stack can be used as data stack too, if it is implemented as RAM
- 22 to 44 Instructions (depending on the configuration)
- Immediate, direct and indirect addressing for all instructions
- Each instruction is executed in 2 to 3 clock cycles
- Operation with slow peripherals and ROMs is possible through separate instruction-wait and data-wait signals
- Codesize up to 1024 instructions
- Memorysize (including peripheral registers) up to 1024 bytes
- Expansion possibilities by changing constants in a package
 - 16bit Datapath
 - up to 64k Codesize
 - up to 64k Memorysize

2.2 Architecture Overview

The following graphic shows only the dataflow. Control signals are not shown. The existence of red registers depends on the configuration of the core. Yellow registers exist in every configuration.



2.2.1 Reset Values

Register	Width	Reset Value
Programmcounter	10	0
Interrupt Enable	1	0
Akkumulator	8	0
Register B	8	0
Register C	8	0
Register D	8	0
Indirect Address	10	0
Stack Pointer	10	0

Table 1: Reset Values of internal Registers

2.3 Instruction Set

2.3.1 Instruction Set Summary

The following table shows all hardware instructions and their runtime in clk cycles.

Inst	#	Dir	@ ¹	C	A	B ²	C ⁹	D ⁹	Description
<i>Load / Store</i>									
LDA <arg>	2	3	3		X				A = Arg
STA <arg>	-	3	3		X				Arg = A
XCHG B ²	2	-	-		X	X			A = B, B = A
XCHG C ⁹	2	-	-		X		X		A = C, C = A
XCHG D ⁹	2	-	-		X			X	A = D, D = A
SIDL ¹	3	-	-						IND [7..0] = A
SIDH ¹	3	-	-						IND [9..8] = A [1..0]
SID <arg> ¹	3	-	-						IND = Arg
CHID <arg> ¹	3	-	-						IND = IND + Arg
SSP <arg> ⁵	3	-	-						SP = Arg
PUSH <arg> ⁵	3	-	-						(SP) = A, SP = SP - 1
POP <arg> ⁵	3	-	-		X				A = (SP + 1), SP = SP + 1
SPID ⁷	3	-	-						IND = SP
PUSHS ⁸	2	-	-						(SysStack) = A
POPS ⁸	2	-	-		X				A = (SysStack)
<i>Arithmetic / Logic</i>									
ADD <arg>	2	3	3	X	X				A = A + Arg
ADDC <arg> ³	2	3	3	X	X				A = A + Arg + C
SUB <arg>	2	3	3	X	X				A = A - Arg
SUBC <arg> ³	2	3	3	X	X				A = A - Arg - C
MUL <arg> ²	2	3	3		X	X			BA = A * Arg
MAC <arg> ⁹	2	3	3		X	X	X	X	DCB = DCB + A * Arg, A = B ^{new}
ROL	2	-	-	X	X				A << through C
ROR	2	-	-	X	X				A >> through C
AND <arg>	2	3	3		X				A = A & Arg
OR <arg>	2	3	3		X				A = A Arg
XOR <arg>	2	3	3		X				A = A ^ Arg
INV	2	-	-		X				Invert A
SWAP	2	-	-		X				A = A [3..0] A [7..4]
<i>Jump</i>									
JMP <arg>	-	2	2 ⁴						Jump to Address
CALL <arg>	-	2	-						Call Subroutine
RET	2	-	-						Return from Subroutine
RETI ¹⁰	2	-	-						Return from Interrupt
<i>Skip</i>									
SKC	3	-	-						Skip if C = 1
SKNC	3	-	-						Skip if C = 0
SKB <arg> ⁶	3	3	3						Skip if A > Arg
SKS <arg> ⁶	3	3	3						Skip if A < Arg
SKEQ <arg>	3	3	3						Skip if A = Arg
SKNEQ <arg>	3	3	3						Skip if A != Arg
<i>Others</i>									
SET C	2	-	-	X	X				C = 1
CLRC	2	-	-	X	X				C = 0
SET IE ¹⁰	2	-	-						Enable Interrupts
CLR IE ¹⁰	2	-	-						Disable Interrupts
CLRA	2	-	-		X				A = 0
CLR MAC ⁹	2	-	-			X	X	X	B = 0, C = 0, D = 0
NOP	2	-	-						No Operation

Table 2: Instruction Set Summary

Immediate Addressing

Dir Direct Addressing

@ Indirect Addressing

Ca Changes Carry

A Changes Akkumulator

B C D Changes Register B C D

- 1 These instructions exist only if indirect addressing is enabled (`indir_on = true`)
- 2 These instructions and register B exist only if multiplication is enabled (`mul_on = true`)
- 3 These instructions exist only if carry arithmetic is enabled (`carry_arith_on = true`)
- 4 The indirect jump is a jump to $PC + \langle Arg \rangle$
- 5 These instructions exist only if user stack is enabled (`user_stack_on = true`)
- 6 These instructions exist only if advanced skips are enabled (`adv_skips_on = true`)
- 7 This instruction exists only if user stack and indirect addressing are both enabled
- 8 This instructions exists only if stack is implemented as RAM (`stack_as_ram = true`) and data on system stack is enabled (`dat_on_sstack = true`)
- 9 These instructions exist only if MAC is implemented instead of mul (`mac_for_mul = true`) and multiplier is enabled (`mul_on = true`).
- 10 These instructions exist only if interrupt support is enabled (`interrupts_on = true`).

When indirect addressing is used, the operand is the value at the memory address IND and after the Instrucion $\langle Arg \rangle$ is addet to IND.

2.3.2 Indirect Addressing

For indirect addressing, the actual content of the indirect addressing register is used as address. After the execution of the instruction, the operands value is added to the indirect addressing register.

Example

```
SID #100 ;IA_Reg = 100
ADD @5 ;Akku = Akku + Value at Adr. 100, IA_Reg = 100 + 5
ADD @-10 ;Akku = Akku + Value at Adr. 105, IA_Reg = 105 - 10
```

2.3.3 Skip Instructions

Every skip instruction skips the following instruction, if the corresponding condition is true. Otherwise the following instruction is executed.

Example

```
LDA #100 ;Akku = 100
SKEQ #101 ;Akku is not 101 --> No Skip
ADD #1 ;Instruction is executed: Akku = 100 + 1
SKEQ #101 ;Akku is 101 --> Skip
ADD #1 ;Instruction is skipped: Akku remains 101
```

2.3.4 Interrupts

The new state of the interrupt enable becomes effective one instruction after the change.

An interrupt is triggered when at least one interrupt line is high.

Each interrupt jumps to his own vector. The vectores are organized as follows:

Interrupt 0	Last instruction in program memory	(0x3FF)
Interrupt 1	Second last instruction in program memory	(0x3FE)
...		

The Interrupt with the highest Index has the highest priority.

Example

```
SET IE ;Enable Interrupts
ADD #1 ;Instruction is executed even if an Interrupt is pending
ADD #1 ;First Instruction which can be interrupted
ORG 3FEh ;
JMP Int1 ;Interruptvector for Interrupt 1 at $3FE = 1022
JMP Int0 ;Interruptvector for Interrupt 0 at $3FF = 1023
```

Akkumulator and Carry are not saved automatically when an interrupt occurs. This must be done by software.

2.3.5 Stack

The stack pointer is decremented after each push-instruction and incremented before each pop instruction.

Example

```
SSP #100 ;SP = 100
PUSH ;Value at Adr. 100 = Akku, SP = 99
POP ;Akku = Value at Adr. 100, SP = 100
```

2.3.6 Multiplication

The multiplier is implemented as unsigned multiplier.

Example (8bit)

0xFF	x	0x04	=	0x03FC0
255	x	4	=	1020

2.3.7 Multiply-Accumulate

The multiply-accumulate instruction is implemented as signed MAC.

Example (8bit)

0x000000	+	0xFF	x	0x04	=	0xFFFFFC
0	+	-1	x	4	=	-4

2.4 Interfaces

This chapter describes the data- and instruction-interfaces. It should give the user the ability to properly connect any external components (ROM, RAM, Peripherals) to the processor core. Therefore the processor is fully vendor and technology independent and can be ported to any device.

Important Note: Only the signal-states at rising edges of *clk* are important in all waveforms

2.4.1 Instruction Interface

Over the instruction interface, the processor reads the programm to execute.

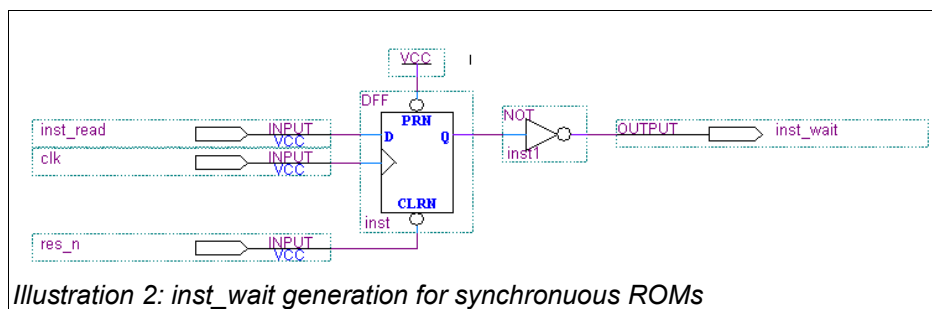
2.4.1.1 General Description

Signal	Dir	Width	Description
clk	-	1	System clock
inst_adr	out	10	Instruction Address
Inst	in	16	Instruction
inst_wait	In	1	Wait, Instruction is not yet valid
inst_read	out	1	Instruction Address changed, Instruction must be updated

Table 3: Signals of the Instruction Interface

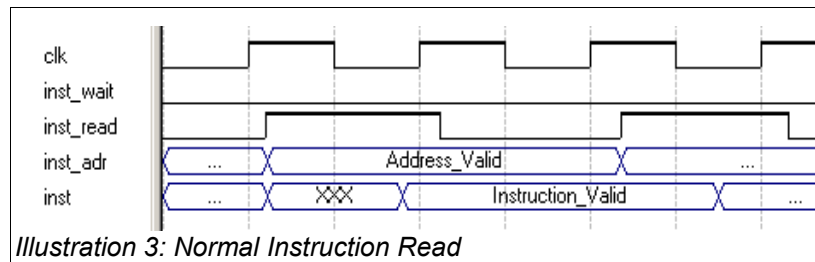
If *inst_adr* is changed (*inst_read* = '1'), the valid instruction must be applied before the next rising edge of *clk*. If this is not possible, *inst_wait* must be set instead of the instruction. *Inst_wait* is cleared again, as soon as the instruction is valid.

In synchronuous ROM-technologies the output is updated only on clock edges. Therefore *inst* will be updated one *clk* cycle too late. In this case *inst_wait* can simply be connected to the inversion of the registered *inst_read* signal as shown below. If this is implemented, the execution times for all instructions is one clock cycle longer than shown in table 2 on page 8.

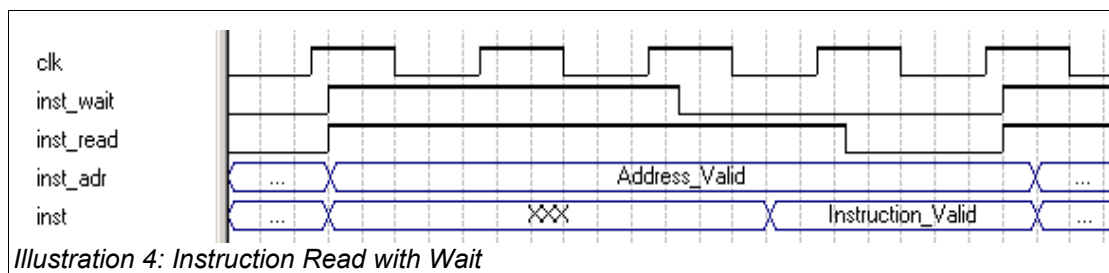


Another possibility to connect synchronuous ROM is to use the inverted clock for the ROM. In this case the ROM must be fast enough to deliever the correct instruction in a half clock cycle. For ROMs in FPGAs this is no problem.

2.4.1.2 Normal Instruction Read¹



2.4.1.3 Instruction Read with Wait¹



¹ Only the signal-states at rising edges of clk are important

2.4.2 Data Interface

Over the data interface, the processor reads and writes the data to process. This is the interface to RAM and peripherals.

2.4.2.1 General Description

Signal	Dir	Width	Description
clk	-	1	System clock
data_adr	out	10	Addressbus
data_out	out	8	Databus to Memory / Peripherals
data_in	In	8	Databus from Memory / Peripherals
wr	out	1	Write Signal
rd	out	1	Read Signal
data_wait	In	1	Wait, Data not yet valid

Table 4: Signals of the Data Interface

If *rd* is high on a rising edge of *clk*, correct data must be applied on *data_in* before the next rising edge of *clk*. If this is not possible, *data_wait* must be set instead. *data_wait* is cleared again, as soon as the valid data is applied to *data_in*.

If *wr* is high on a rising edge of *clk*, *data_out* must be interpreted. If it is required that *data_out* remains unchanged, *data_wait* must be set additionally. As soon as *data_out* can change again, *data_wait* must be reset.

The described interface is fully compatible with normal synchronuous RAM. To preserve this compatibility, *data_in* must be multiplexed asynchronous depending on *data_adr*.

Chip select signals for peripherals can be generated through simple combinatorial logic out of *data_adr*.

2.4.2.2 Normal Data Read¹

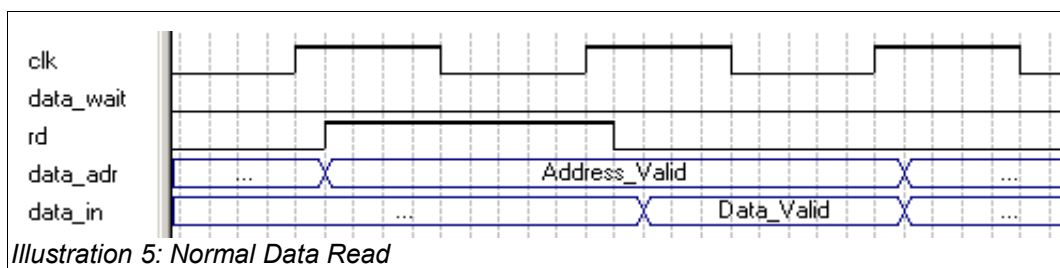
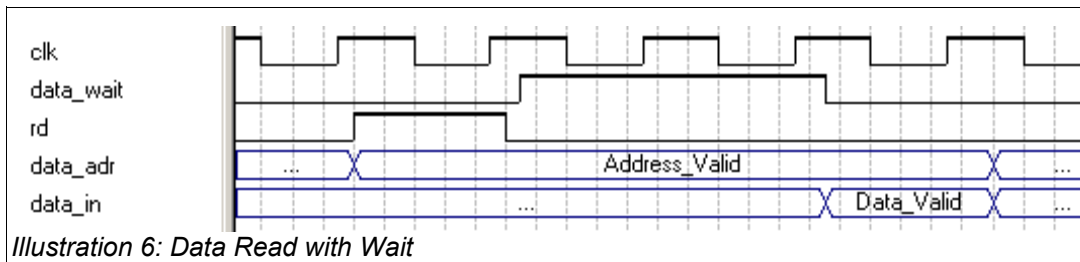


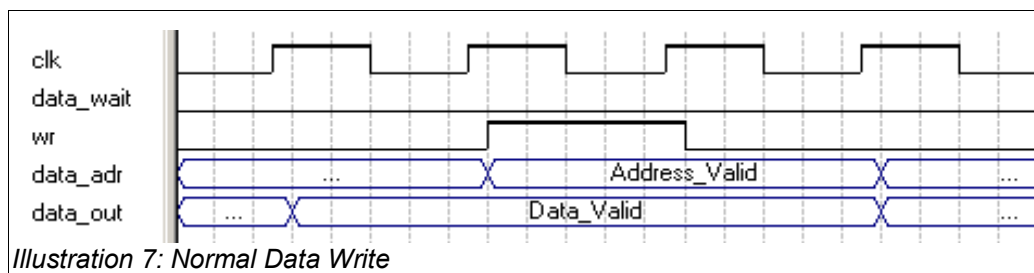
Illustration 5: Normal Data Read

¹ Only the signal-states at rising edges of *clk* are important

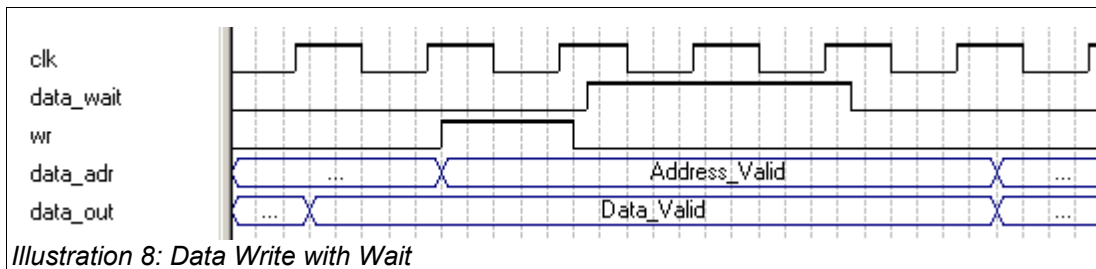
2.4.2.3 Data Read with Wait¹



2.4.2.4 Normal Data Write¹



2.4.2.5 Data Write with Wait¹



2.4.3 Interrupts

If at least one signal of the *interrupts* vector is high, an interrupt is triggered. Interrupts must stay high, until they are cleared by software. If you don't need interrupts, connect the interrupt port to a logical low.

2.4.4 Debug Interface

The debug interface consists of three signals:

- rx RS-232 data line from debugger to computer (115'200 Baud)
- tx RS-232 data line from computer to debugger (115'200 Baud)
- res_n_dbg Is low if either res_n is low or a reset-command is applied to the debugger. So you can apply the reset-command from the debugger to peripherals too.

If the debugger is disabled, connect rx to logical '1' and leave the other signals open.

¹ Only the signal-states at rising edges of clk are important

2.5 How to configure the Core

This Core can be configured in two different Ways:

- Generics
 - Instruction set configuration
 - Stack implementation (FF-Array or RAM)
 - Instructon stack depth
 - Interrupts support / number of Interrupts
- Changes in the package panda_pkg.vhdl
 - Instruction-bus width
 - Instruction-address width
 - Data-bus width
 - Data-address width

2.5.1 Generics

Generics are the easiest way to configure some fundamental settings. In the following chapters describe the different generics and their effects on the core.

If some instructions are switched off, take care that they are not used in the programcode. The assembler does not react on disabled instructions!

2.5.1.1 *stack_depth*

Type: Integer

Value: Default: 6 Minimum: 1 Maximum: -

Effect: Sets the depth of the Stack. If depth is 6 for example, the maximum of nested subroutine calls is 6 (or 5 and 1 interrupt). Take also care of subroutine calls in interrupts!

2.5.1.2 *num_of_ints*

Type: Integer

Value: Default: 1 Minimum: 1 Maximum: -

Effect: Sets the number of interrupt inputs. For more information have a look at chapter 2.3.4 Interrupts.

2.5.1.3 *interrupts_on*

Type: Boolean

Default: true

Effect: If this generic is „true“, interrupts are supported. If it is off, the interrupts inputs have no effect. If this generic is false, connect the interrupt line to a logical low.

True --> Interrupts are supported, CLR IE, SET IE, RETI are available

False --> Less ressource usage

2.5.1.4 indir_on

Type: Boolean

Default: true

Effect: If this generic is „true“ all indirect addressing instructions are enabled, otherwise indirect addressing is disabled.

True --> Indirect addressing, SID H, SID L, SID # and CHID are available
SPID is only available if user stack is enabled too.

False --> Less resource usage

2.5.1.5 carry_arith_on

Type: Boolean

Default: true

Effect: If this generic is „true“ the carry arithmetic instructions (ADDC, SUBC) are enabled, otherwise they are disabled. If minimal resource usage has priority, they can be disabled and replaced by a sequence of other instructions as described below.

True --> ADDC and SUBC are available

False --> Less resource usage

Replacement of ADDC <arg>

```
ADD <arg> ;Addition
SKNC      ;Add Carry if required
ADD #1    ;..
```

2.5.1.6 adv_skips_on

Type: Boolean

Default: true

Effect: If this generic is „true“, advanced skip instructions (SKB, SKS) are enabled. If minimal resource usage has priority, they can be disabled and replaced by sequences of other instructions as described below.

True --> SKB and SKS are available

False --> Less resource usage

Replacement of SKB <arg>

```
STA tmp    ;Save Accumulator
LDA <arg>   ;Compare Arg and Accumulator
SUB tmp    ;..
LDA tmp    ;Restore Accumulator
SKC        ;Skip if required
```

2.5.1.7 *mul_on*

Type: Boolean

Default: false

Effect: If this generic is „true“ and *mac_for_mul* is „false“ the single cycle multiplier is enabled. This generic also enables the implementation of the register B. The multiplier requires a lot of device resources, if there are no dedicated multipliers on the chip.

True --> MUL and XCHG B are available (Recommended for FPGAs)

False --> Less resource usage (Recommended for CPLDs)

2.5.1.8 *mac_for_mul*

Type: Boolean

Default: false

Effect: If this generic is „true“ and *mul_on* is „true“ the MAC-Unit is implemented. This generic also enables the implementation of the register B, C and D. The MAC requires a lot of device resources, if there are no dedicated multipliers on the device. If MAC is enabled, MUL is disabled. The MAC instruction can be used instead.

True --> MAC, CLR MAC, XCHG B, XCHG C, XCHG D are available

False --> Less resource usage (Recommended for CPLDs)

2.5.1.9 *user_stack_on*

Type: Boolean

Default: false

Effect: If this generic is „true“, the user stackpointer and the corresponding instructions are implemented. The user stack is a stack for data in the RAM address space.

True --> SSP, PUSH and POP are available.
SPID is only available if indirect addressing is turned on too.

False --> Less resource usage

2.5.1.10 *stack_as_ram*

Type: Boolean

Default: false

Effect: If this generic is „true“ the system stack is implemented as a RAM with an addresspointer. Otherwise the Stack is implemented as a FF-Array. The stack should *only* be implemented as RAM, if there are RAM-resources on the chip. If the stack is implemented as RAM, *stack_depth* should be a power of two.

True --> Stack is implemented as RAM (Recommended for FPGAs)

False --> Stack is implemented as FF-Array (Recommended for CPLDs)

2.5.1.11 *dat_on_sstack*

Type: Boolean

Default: false

Effect: If this generic is „true“ and the system stack is implemented as RAM (*stack_as_ram = true*) , the accumulator can be pushed on the system stack. This is not very efficient, because the system stack is normally wider than the accumulator but this is no problem, because the RAM has more space than needed for calls and interrupts anyway. Therefore the system stack can replace the user stack and therefore save some device ressources. Note that data on the system stack is not accessible with direct and indirect addressing in contrast to data on the user stack.

True --> Data can be pushed and popped to and from the system stack
PUSHS and POPS are available

False --> System stack is reserved for call- and interrupt-return addresses

2.5.1.12 *debug_on*

Type: Boolean

Default: false

Effect: If this generic is „true“, the on-chip-debugger is implemented. For more Information, look at chapter 4.4 On-Chip-Debugger. The Debugger needs a lot of ressources, so it should only be enabled during development and debugging.

True --> On-Chip-Debugging is available

False --> Less ressource usage

2.5.1.13 *clk_frequency*

Type: Integer

Default: 10'000'000

Effect: This generic must be set to the clock frequency in herz. It used to generate the correct baudrate of 115'200 for the RS-232 debugging interface.

2.5.2 Package panda_pkg.vhdl

If the resource usage is a critical point, the core can be further optimized by changing the width of the address buses. If you change any values, make sure that the system stack width and biggest register width have still the right value. Don't change anything else in this package!

2.5.2.1 Databus Address Width

Name of Constant: data_adr_width

Default: 10

Maximum: 10

Minimum: 1

Notes: If the value is smaller than 8, the instruction „SID H“ becomes useless.
If an address in the programcode is bigger than possible through this bus width, the assembler uses the least significant part of the address.

2.5.2.2 Instruction Address Width

Name of Constant: inst_adr_width

Default: 10

Maximum: 10

Minimum: 1

Notes: This setting changes the interrupt vectors too.
A smaller instruction address width reduces the resource usage of the system stack, especially if it is implemented as FF-array.

2.5.2.3 System Stack Width

Name of Constant: sysstack_width

Default: inst_adr_width

Possible Values: inst_adr_width
data_width

Notes: This constant must be set to the bigger one of both possible values.

2.5.2.4 Biggest Register Width

Name of Constant: biggest_reg_width

Default: inst_adr_width

Possible Values: inst_adr_width
data_width
data_adr_width

Notes: This constant must be set to the biggest one of all three possible values.

2.5.3 Expanding the Core to 16-Bit Wordwidth

To expand the core to 16-Bit Wordwidth some more constants in the *panda_pkg.vhdl* package must be changed and the assembler must be told to use the bigger instruction width (24bit instead of 16bit).

Of course this changes also the width of some internal registers (e.g. indirect addressing register). So the values in this documentation are no more true (they are true for the 8-bit version of the controller).

2.5.3.1 Constants in *panda_pkg.vhdl*

The following table shows how to configure the constants for 16-Bit wordwidth.

Constant	New Value
inst_width	24
data_width	16
data_adr_width	1..16
inst_adr_width	1..16

Table 5: Constant Values for 16-Bit

For more information about these constants and how to configure the other constants, please refer to 2.5.2 Package *panda_pkg.vhdl*.

2.5.3.2 Register Widths

If you expand the core to 16bit datapath width, the internal registers have the following widths:

Akkumulator, B, C, D: 16bit

Indirect adress, Stackpointer: *data_adr_width* (constant in *panda_pkg.vhdl*)

2.5.3.3 Assembler

Just change the first line in you source file from *#CPU PANDA* to *#CPU PANDA24* (for 24-bit instruction width).

The assembler creates a .hex file with 32-bit wordwidth, because .hex files can only have 8, 16 or 32-bit wordwidth. You have to truncate the first 8 bits of every word with a hex-editor or use the romwriter program which truncates the first 8 bits automatically.

2.6 Connecting Peripherals

This chapter describes how to connect peripherals to the controller to build a whole system on chip.

2.6.1 Instruction ROM

If an asynchronous ROM is used, it should be connected as shown in the following table:

ROM Port	PANDA Port
data_out	inst
address_in	inst_adr
LOGIC LOW	inst_wait
-	inst_read

Table 6: Rom Connection

For using a synchronuous ROM look at 2.4.1 Instruction Interface.

2.6.2 Data Bus Block

The signals *wr*, *rd*, *data_adr* and *data_out* can be connected directly to all peripherals. Data from peripherals to the controller (*data_in*) must be multiplexed and chip select signals for peripherals must be generated in a bus block. This is described in the following chapter.

The connection of the *data_wait* signal depends on the generated wait signal from the peripherals. If you have slow peripherals and use this signal, please refer to 2.4.2 Data Interface.

2.6.2.1 data_in Multiplexer

Asynchronous

To achieve full speed, the *data_in* multiplexer has to be asynchronous as shown in the following code sample:

```
data_in_cpu <= data_out_ram WHEN data_adr_cpu(9 DOWNT0 8) = "00" ELSE
               data_out_per1 WHEN data_adr_cpu(9 DOWNT0 7) = "010" ELSE
               data_out_per2;
```

Synchronuous

If paths from peripherals to the controller become too slow, a synchronuous multiplexer can shorten them. This results in one clock cycle longer read instructions (all instructions with direct or indirect addressing). It can be implemented as shown in the following code sample:

```
din_mux : PROCESS (clk, res_n)
BEGIN
    IF res_n = '0' THEN
        data_in_cpu <= (OTHERS => '0');
        data_wait <= '0';
    ELSIF rising_edge(clk) THEN
        CASE data_adr_cpu(9 DOWNT0 7) IS
            WHEN "000" | "001" => data_in_cpu <= data_out_ram;
            WHEN "010"         => data_in_cpu <= data_out_per1;
            WHEN OTHERS        => data_in_cpu <= data_out_per2;
        END CASE;
        data_wait <= rd_cpu;
    END IF;
END PROCESS din_mux;
```

2.6.2.2 Chip Select Signals

The chip select signals must be decoded asynchronously.

```
cs_per1_n <= '0' WHEN data_adr_cpu(9 DOWNT0 7) = "010" ELSE
              '1';
cs_per2_n <= '0' WHEN data_adr_cpu(9 DOWNT0 7) = "011" OR data_adr_cpu(9) = '1' ELSE
              '1';
```

2.6.2.3 Data RAM Write Signal

The write signal for the data RAM must be generated separately, because the RAM blocks in FPGAs often don't have a chip select signal.

```
wr_ram <= wr_cpu WHEN data_adr_cpu(9 DOWNT0 8) = "00" ELSE
           '0';
```

2.6.3 Own Peripherals

Own peripherals can be connected to the processor and bus block as shown in the following table.

The connection of the *data_wait* signal depends on the generated wait signal from the peripherals. If you have slow peripherals and use this signal, please refer to 2.4.2 Data Interface.

Peripheral Port	PANDA Port	Bus Block Port
data_in	data_out	-
data_out	-	data_out_perX
data_adr	data_adr	-
wr_in	wr	-
rd_in	rd	-
cs_n	-	cs_perX_n

Table 7: Peripheral Connection

2.7 Files

The design contains the following files:

panda_top.vhd	Top-module of the processor (std_ulogic I/Os)
panda_top_stdlv.vhd	Top-module of the processor (std_logic I/Os)
panda_ctrl.vhd	Control unit of the processor
panda_bus.vhd	Bus unit of the processor
panda_alu.vhd	Arithmetic-logic unit of the processor
panda_pkg.vhd	Panda package
panda_debugger.vhd	Debugger
panda_debug_control.vhd	Control unit of the debugger
panda_comport.vhd	RS-232 interface for the debugger
ASCII_CODE.vhd	ASCII-Package for the debugger

2.8 Synthesis Results

For each synthesis run all activated features are listed. Unlisted features are deactivated. Synthesis is optimized for Area, not for Speed.

2.8.1 Altera

2.8.1.1 MAX II CPLD (EPM1270GF256C3)

Small

Stackdepth: 1
No interrupts supported

LE's: 158
Fmax: 114MHz

Medium

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 1 Carry Arithmetic

LE's: 322
Fmax: 81MHz

Large

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier

LE's: 526
Fmax: 67MHz

Debug

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Debugger

LE's: 671
Fmax: 76MHz

2.8.1.2 Cyclone II FPGA (EP2C5Q208C7)

Small

Stackdepth: 1
No interrupts supported

LE's: 169
RAM-Bits: 0
Multipliers: 0
Fmax: 131MHz

Medium

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 1 Carry Arithmetic

LE's: 326
RAM-Bits: 0
Multipliers: 0
Fmax: 98MHz

Large

Stackdepth: 256 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier Stack as RAM Data on System Stack

LE's: 446
RAM-Bits: 2560
Multipliers: 1
Fmax: 87MHz

Debug

Stackdepth: 256 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier Stack as RAM Data on System Stack
Debugger

LE's: 752
RAM-Bits: 2560
Multipliers: 1
Fmax: 88MHz

2.8.2 Xilinx

2.8.2.1 Spartan3 FPGA (XC3S200)

Small

Stackdepth: 1
No interrupts supported

Slices: 105
RAM-Bits: 0
Multipliers: 0
Fmax: 59MHz

Medium

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 1 Carry Arithmetic

Slices: 241
RAM-Bits: 0
Multipliers: 0
Fmax: 59MHz

Large

Stackdepth: 256 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier Stack as RAM Data on System Stack

Slices: 275
RAM-Bits: 2560
Multipliers: 1
Fmax: 51MHz

Debug

Stackdepth: 256 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier Stack as RAM Data on System Stack
Debugger

LE's: 464
RAM-Bits: 2560
Multipliers: 1
Fmax: 57MHz

2.8.3 Lattice

2.8.3.1 MachXO CPLD (LCMXO2280E-5FT256C)

Small

Stackdepth: 1
No interrupts supported

LUT's: 142
Fmax: 102MHz

Medium

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 1 Carry Arithmetic

LUT's: 306
Fmax: 77MHz

Large

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier

LUT's: 403
Fmax: 70MHz

Debug

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Debugger

LE's: 577
Fmax: 73MHz

2.8.3.2 XP2 FPGA (LFXP3E-4208C)

Small

Stackdepth: 1
No interrupts supported

LUT's: 150
RAM-Bits: 0
Fmax: 88MHz

Medium

Stackdepth: 6 Indirect Addressing Advanced Skips
Interrupts: 1 Carry Arithmetic

LUT's: 284
RAM-Bits: 0
Fmax: 86MHz

Large

Stackdepth: 256 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier Stack as RAM Data on System Stack

LUT's: 324
RAM-Bits: 2560
Fmax: 71MHz

Debug

Stackdepth: 256 Indirect Addressing Advanced Skips
Interrupts: 4 Carry Arithmetic User Stack
Multiplier Stack as RAM Data on System Stack
Debugger

LUT's: 552
RAM-Bits: 2560
Fmax: 71MHz

3 Software Installation Guide

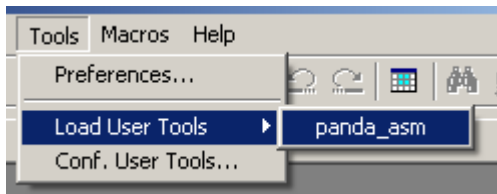
This chapter explains the installation of the software tools you need to program the Panda.

3.1 Assembler Installation

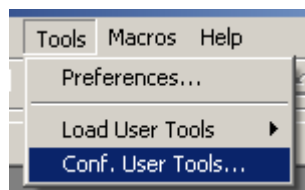
1. Copy the folder `<PandaRoot>\ASSEMBLER\HASM` to your hard-disk.

3.2 Editor Installation

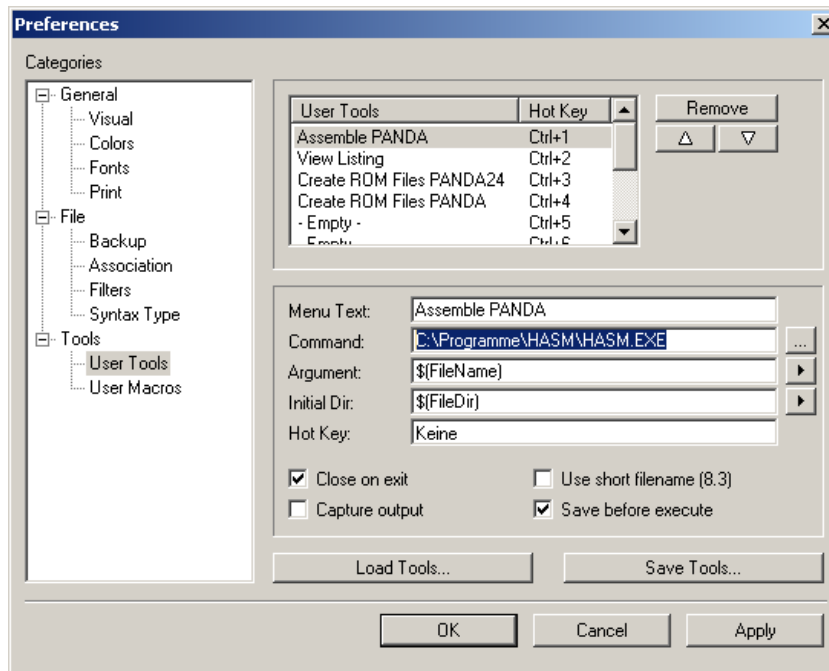
1. Execute the file `<PandaRoot>\EDITOR\cedt370r.exe`.
This will install Crimson Editor on your Computer.
2. Copy the file `<PandaRoot>\EDITOR\extensions.asm` to the directory `<CrimsonEditorRoot>\link`.
3. Copy the following files to the directory `<CrimsonEditorRoot>\spec`:
`<PandaRoot>\EDITOR\panda.key`
`<PandaRoot>\EDITOR\panda.spc`
4. Copy the file `<PandaRoot>\EDITOR\panda_asm.cmd` to the directory `<CrimsonEditorRoot>\tools`.
5. Start CrimsonEditor.
6. To install the panda_asm tools, click the following menu item.



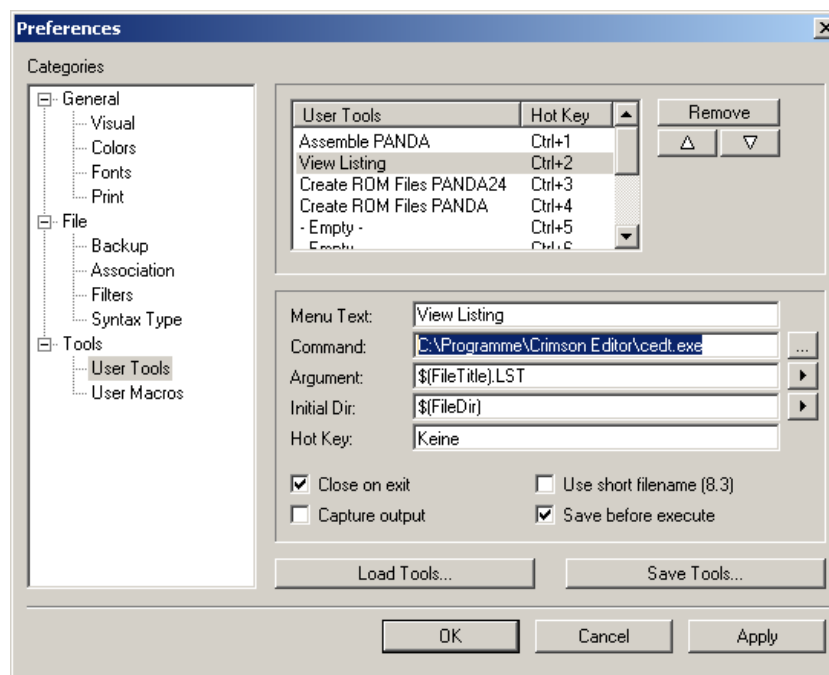
7. Now start the tools configurator.



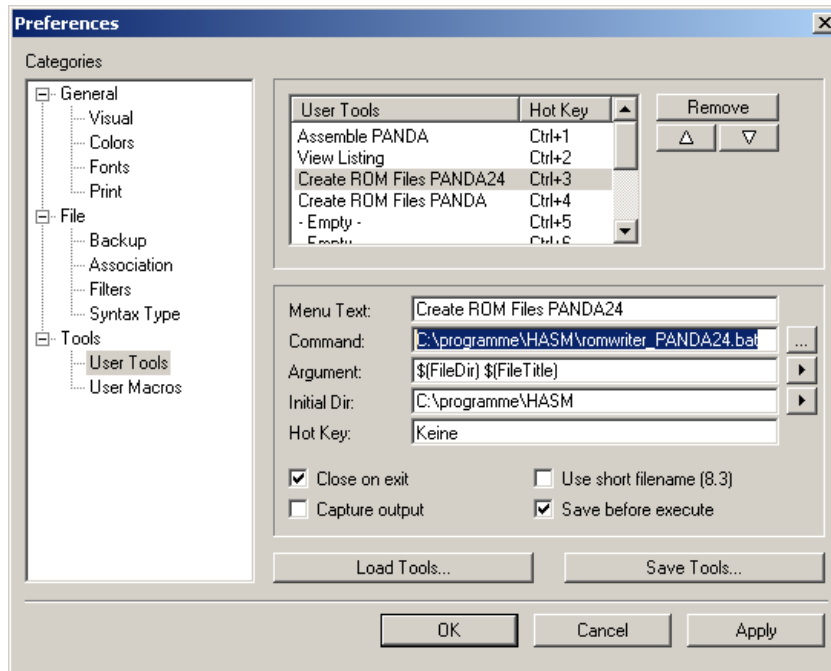
8. Now set the „Command“-entry of „Assemble PANDA“ to `<HASMRoot>\HASM.EXE` (3.1 Assembler Installation) from of your installation.



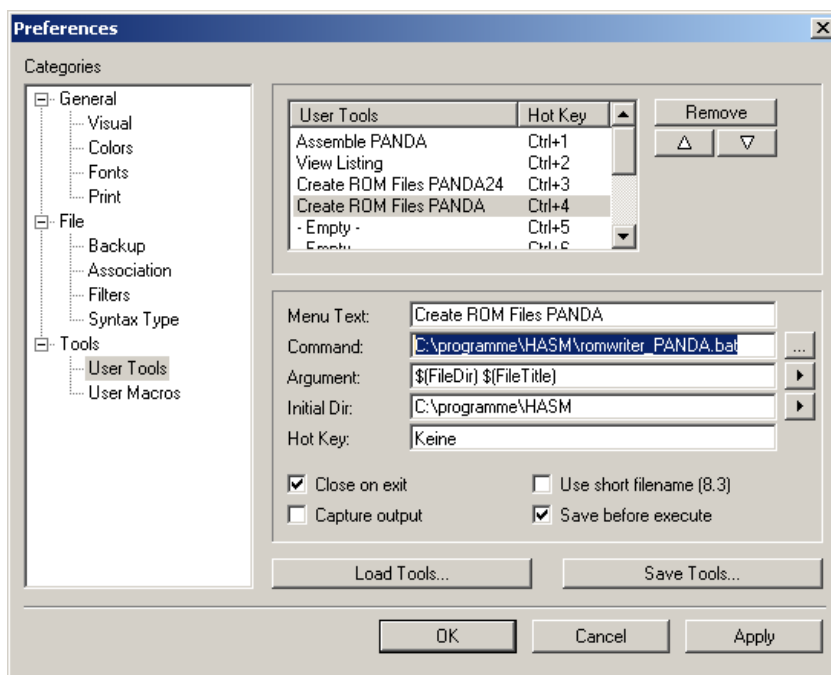
9. Set the „Command“-entry of „View Listing“ to `<CrimsonEditorRoot>\cedt.exe` from your installation.



10. Set the „Command“-entry of „Create ROM Files PANDA24“ to `<HASMRoot>\romwriter_PANDA24.bat` from your installation.
Set the „Initial Dir“-entry of „Create ROM Files PANDA24“ to `<HASMRoot>`



11. Set the „Command“-entry of „Create ROM Files PANDA“ to `<HASMRoot>\romwriter_PANDA.bat` from your installation.
Set the „Initial Dir“-entry of „Create ROM Files PANDA“ to `<HASMRoot>`



4 Software User Guide

This chapter shows how to implement an assembler-programm for Panda.

4.1 Software development Flow

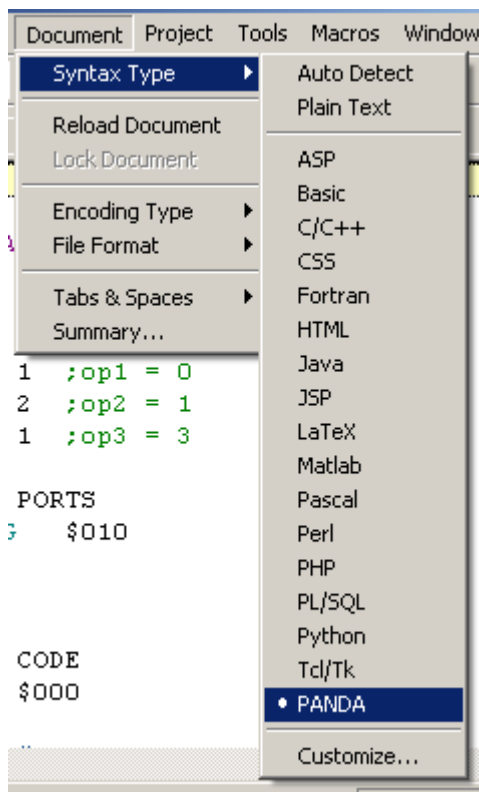
1. Write assembler source file in Crimson Editor
2. Assemble source file with HASM (tools menu in Crimson Editor)
3. View the list file and check it for errors (tools menu in Crimson Editor)
4. Create vendor dependent- and VHDL-ROM files with the romwriter tool (tools menu in Crimson Editor)
5. Program the device and test/debug the code in system

4.2 Editor User Guide

To follow the different steps of this chapter, please start Crimson Editor and open the file `<HASMRoot>\test.ASM` from your installation.

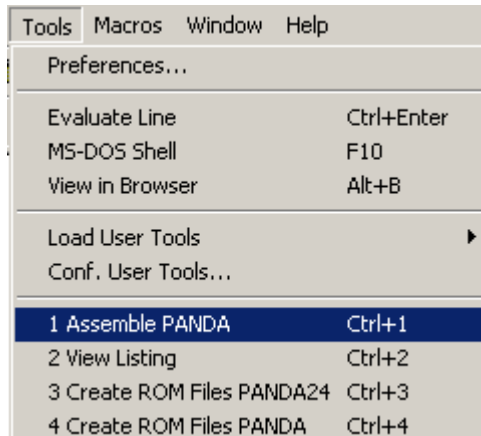
4.2.1 Enabe Syntax Highlighting

To enable syntax highlighting click the following meue item:



4.2.2 Assemble a Source-File

Use the tools menu or press „Ctrl+1“ to start the assembler:

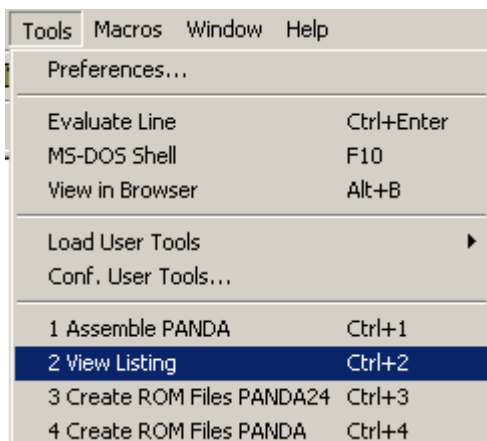


The corresponding Intel .hex File to your assembler programm will now be created. You can find it under the same path as the .asm-file under *<FileName>.hex*.

The corresponding List file is also under the same path and named *<FileName>.lst*.

4.2.3 View the Listing

Use the tools menu or press „Ctrl+2“ to view the List-File from the assembler.



Check the list file for errors. Errors are marked above the codeline they occurred. At the end of the list file, the total number of errors is noted.

You can enable syntax highlighting again as described in 4.2.1 Enable Syntax Highlighting.

4.2.4 Creating ROM-Files

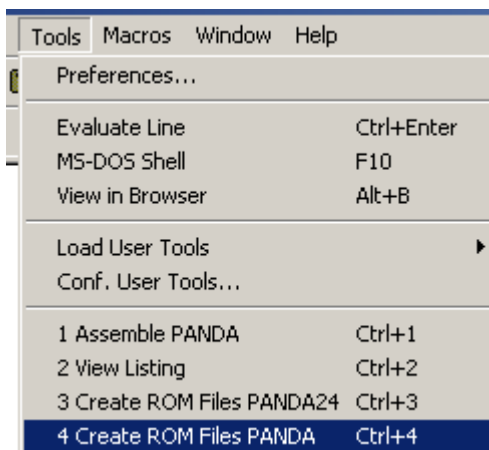
The romwriter tool creates adequate ROM-initialization files in the following formats:

.mif	Altera and Xilinx Block-Rom initialization
.coe	Xilinx Block-Rom initialization
.mem	Lattice Block-Rom initialization
.vhd	Asynchronous and synchronous ROM in VHDL for simulation and implementation in logic cells

All these files are created in the same directory as the <filename>.asm file is in. They are named as follows:

<filename>.mif
<filename>.coe
<filename>.mem
AsyncVHDL_<filename>.vhd
SyncVHDL_<filename>.vhd

Use „Create ROM Files PANDA24“ only if you changed the constants in panda_pkg.vhdl and the instruction with is now 24bits. Otherwise use „Create ROM Files PANDA“.



4.3 Assembler Syntax

Let's go through at the file `<HASMRoot>\test.ASM` from your installation and have a look at the different parts of it and their functionality.

4.3.1 Definitions and Directives

Firstly the assembler needs to know for which processor it should generate code. This is done with the following directive in the first line of the file:

```
#CPU PANDA
```

If you want to include files, you can do this with the include-directive. The following example shows how to include the File „PANDAMAC.INC“ which is in the same folder as the .asm file.

```
#INCLUDE "PANDAMAC.INC"
```

The next command tells the assembler to create a list of all used labels in the .lst file. If you don't want this label list, just leave this directive away.

```
#SYMLIST
```

4.3.2 Basic Conventions

4.3.2.1 Comments

Everything after a semicolon is interpreted as comment. If you want to write longer comments, you have to start each line with a semicolon.

4.3.2.2 Number Formats

All numbers are interpreted as decimal normally. Hexadecimal numbers must end with a „h“. Binary numbers must end with a b.

```
200      ;Decimal Number
20h      ;Hexadecimal Number 20 = 32(Decimal)
1010b    ;Binary Number 1010 = A(Hexadecimal) = 10(Decimal)
```

4.3.2.3 Operands for fixed Numbers

For calculations the following operands can be used:

^	Bitwise Exor	*	Multiplication
/	Division	%	Modulo
-	Subtraction	+	Addition
<<	Shift Left	>>	Shift Right
&	Bitwise And		Bitwise Or

Just an example to show the use of such operands:

```
OR      #1<<4      ;Set bit 4 in Akku
```

4.3.2.4 Segments

Code segments are indicated with the CSEG-directive, data segments with the DSEG-directive.

4.3.3 Declaring Peripheral Registers

Each peripheral should be defined as data segment at a specified address. Use the following commands for this:

```
DSEG      <PeripheralName>
REGORG    <Address>
```

It is important to use the REGORG directive and not the ORG-directive for datasegments. Otherwise the assembler will generate errors.

For every peripheral register one word of memory must be reserved. This is done through the following command:

```
<RegName> RMW      1          ;Reserve 1 Word for <RegName>
```

Its also possible to reserve more than one word. This is used for peripherals which contain memory. To do this, just replace the „1“ with the memory size.

```
<Label>    RMW      256        ;Reserve 256 Words for <Label>
```

If you want to use the same address for different register names (e.g. different input and output ports), just leave away the RMW-directive until the last used register-name.

```
<RegName1>
<RegName2> RMW      1          ;Reserve 1 Word for <RegName1> and <RegName2> together
```

It is also possible to define the address of each register as constant with the EQU-directive. This is not recommended, because the assembler does not generate errors if some addresses are used twice.

```
<RegName> EQU      <Address>
```

4.3.4 Declaring Variables

Variables should be defined as data segment in the RAM-addressspace. The different variables are then defined the in the same way as the peripheral registers.

```
                DSEG      VARIABLES
                REGORG    <RAM-Startaddress>
<Var1>         RMW      1          ;Reserve 1 Word for <Var1>
<VarSpace>     RMW      8          ;Reserve 8 Words for <VarSpace>
```

4.3.5 Declaring Constants

Constants are defined with the EQU-directive.

```
<ConstName>    EQU      <Value>
```

4.3.6 Macros

For small codesegments which are often used, you can define a macro. Afterwards the macro can be used as any other operator and the assembler replaces it with the defined instructions.

Macros are only used for codesements which are so small that it makes no sense to implement them as subroutine. Bigger codesements should be implemented as subroutines, because the macro needs the full codespace everytime it is used. The subroutine only needs the codespace once and a call instruction everytime it is used.

A macro can be defined as shown in the following example:

```
<MacroName>    MACRO
                <Instructions>
                ENDMACRO
```

To use the macro, just write its name as an instruction and the assembler will replace it by the defined instructions.

4.3.7 Code Syntax

In all examples of the following chapter, the LDA instruction is used. For a complete list of all instruction please refer to 2.3.1 Instruction Set Summary.

Each line can begin with label (used as jump- and call-target) followed by the instruction and, depending on the instruction, an operand. A line can also only contain a label without instruction or an instruction without a label. In the second case the label must be replaced by a whitespace.

```
<label1>                ;Line with label but no instruction
<label2>    LDA  <operand>    ;Line with label and instruction
                LDA  <operand>    ;Line with instruction but no label
```

The operand can be a register, a variable or an address as number for direct addressing. The usage of numbers is not recommended, because it is bad coding style.

```
LDA  <RegisterName>
LDA  <VariableName>
LDA  5                ;Not recommended
```

The operand is a fix number for instructions with immediate values. To indicate immediate values the route symbol is used.

```
LDA  #100                ;Hex or binary values are possible too
```

Indirect addressing is indicated by an @-symbol. The indirect addressing register is changed by the number after the @-symbol after the execution of the instruction. This number can be positive or negative.

```
LDA  @1                ;Load Akku with the Value at the Address specified in the
                        ;..ind. Addr. Reg., increment ind. Addr. Reg afterwards
```

When using indirect jumps, the value 0 after the at-symbol specifies a jump to the actual line. Positive values indicate forward jumps, negative values indicate backward jumps.

```
Label1    LDA  #1
          JMP  @2                ;Jump to Label3
Label2    LDA  #2
Label3    JMP  @-3               ;Jump to Label1
```

If you want some code to be located at a fixed address, use the ORG directive. Use this directive only if you really need to know the exact address of some code (e.g. Interrupt Vectors).

```
ORG  3FFh
```

4.3.8 Interrupts

The interrupt Vectors are located at the end of the program memory. Normally the Interrupt0 is at the address 0x3FF. If you make the instruction address bus smaller (see 2.5.2.2 Instruction Address Width) the interrupt vectors are changed too. For more Information see chapter 2.3.4 Interrupts.

```

CSEG Interrupts
ORG 3FEh
IntVect1 JMP <InterruptServiceRoutine1> ;Interrupt Vector 1 at Address 0x3FE
IntVect0 JMP <InterruptServiceRoutine0> ;Interrupt Vector 0 at Address 0x3FF

```

4.3.9 Include File

You can find the include file *PANDAMAC.INC* in the directory *<HASMRoot>* of your installation. To use the file, copy it in the same directory as your .asm file is.

This file includes some useful macros such as conditional jumps and 16-bit arithmetics.

These macros are identified by the syntax highlighter and shown in a different color than normal instructions.

Before you use this file, you should check whether all instructions used by the *PANDAMAC.INC* file are enabled in your HW-implementation of the controller. If they are not, you cannot use the corresponding macros or you have to change them.

4.4 On-Chip-Debugger

A simple on-chip-debugger with RS-232 interface is available, if the corresponding generic is set. Due to its ASCII commands and responses, it can be controlled with a simple terminal program (e.g. HyperTerminal).

4.4.1 Hardware Connections

The connection settings of the RS-232 connection to the PC are described in the following table:

Setting	Value
Baudrate	115200
Databits	8
Parity	None
Stop-bits	1
Flow-control	None

Table 8: Debugger RS-232 Connection Settings

The signals *rx* and *tx* must be converted to RS-232 voltage level. For this, you can use for example the MAX232 device. Then you can connect the converted signals to your computer (*rx* : PC --> Debugger, *rx* : Debugger --> PC).

4.4.2 Commands

Each command ends with a carriage return. To send the command „b0ff“ for example you have to type „b0ff<Enter>“. All commands are written in small letters.

Command	Parameter	Description
r	-	Run
b<param>	Address of Breakpoint in Hex	Set Breakpoint
e<param>	1 -> Enable Breakpoint 0 -> Disable Breakpoint	Enable / Disable Breakpoint
s	-	Single Step
h	-	Halt
x	-	Reset

Table 9: Debugger Commands

4.4.3 Interpreting the Response

The debugger sends the current state of all internal registers to the computer, each time the controller is stopped. So the current state is sent after every *Single Step*, *Halt* or *Reset* command and when the breakpoint is reached (if enabled). It is also sent after a HW-reset.

The current state of all internal registers which are implemented is sent as shown in the following Table. The values of different registers are separated with white spaces. The response of the debugger ends with a carriage return. Not implemented registers will not be sent. Therefore the response of the debugger depends on the current settings of the generics.

Register	Hex-Nibbles	Sent if
Programmcounter	3	-
Akkumulator	2	-
Carry	1	-
Interrupt Enable	1	interrupts_on = true
Indirect Adr.	3	indir_on = true
Register B	2	mul_on = true
Register C	2	mul_on = true, mac_for_mul = true
Register D	2	mul_on = true, mac_for_mul = true
Stack Pointer	3	user_stack_on = true

Table 10: Response of Debugging Module